

PIKE: Evidence-Guided Retrieval for Email Question Answering

Anonymous ACL submission

Abstract

Personal email question answering often requires recovering evidence from incomplete recollections and connecting information across multiple conversations. Existing benchmarks emphasize single-message lookup, leaving these broader information needs less explored. We introduce a benchmark of 640 evidence-grounded questions across 50 Enron and Avocado inboxes, covering six search intents from keyword lookup to event progression, action tracking, and recurring roles. We also present PIKE, an email-specific framework that combines content retrieval with structured person, topic, temporal, and broadcast signals. An evidence-aware critic assesses coverage and issues targeted requeries, after which the system selects evidence and generates an answer. Dynamic PIKE achieves 85.3% Recall@15, compared with 80.6% for BM25, using 1.10 retrieval rounds on average. Gains are strongest for recurring-role queries and persist across three evaluated LLM backends. Ablations reveal query-dependent contributions from person and temporal retrieval, while a rubric-based audit identifies missed evidence gaps as a limitation of critic stopping decisions. Holding retrieved candidates fixed, aspect-based selection and generation improves answer completeness from 75.6% to 81.0% relative to direct generation, while maintaining high citation faithfulness. Together, these findings clarify when structured retrieval helps personal email QA and why finding evidence, recognizing its sufficiency, and using it effectively require distinct evaluation.

1 Introduction

Email remains one of the most widely used communication tools in professional settings (Bendersky et al., 2022), serving as a long-term archive of decisions, discussions, and organizational knowledge. For many users, their inbox becomes a substantial personal knowledge base accumulated over

years of communication. While conversational search is receiving increasing attention (Mo et al., 2025), personal email question answering beyond single-message lookup remains comparatively underexplored. Existing work provides important foundations for email search and personalized retrieval (Bendersky et al., 2022; Ryan et al., 2025), but less attention has been paid to questions involving incomplete memories or information distributed across conversations. For example, a user might ask, “How did the project deadline change, and what still needs to be done?” Answering may require connecting updates, decisions, and outstanding tasks across multiple threads. Personal email QA thus extends beyond locating a matching message: it requires recovering and organizing evidence across conversations.

EnronQA (Ryan et al., 2025) establishes an important foundation for this setting through a large-scale benchmark organized by user, with single-hop questions grounded in individual emails. In its reported retrieval experiment, BM25 (Robertson and Zaragoza, 2009) achieves 87.5% Recall@5 without query rewriting. The authors attribute this strong performance in part to lexical overlap between queries and email contents, particularly names and events retained through specificity-oriented question generation. This result motivates a complementary research question: *Example: When is lexical retrieval sufficient for personal email QA, and when do incomplete recollections, cross-thread evidence, and temporal or relational information needs benefit from additional retrieval structure?* Answering this question requires a benchmark that varies both the specificity of query cues and the evidence needed to answer them.

We investigate this question through an evidence-grounded benchmark and PIKE (Personal Inbox Knowledge-graph Email Retrieval), an email-specific RAG framework. Email offers complementary retrieval cues: who participated, when

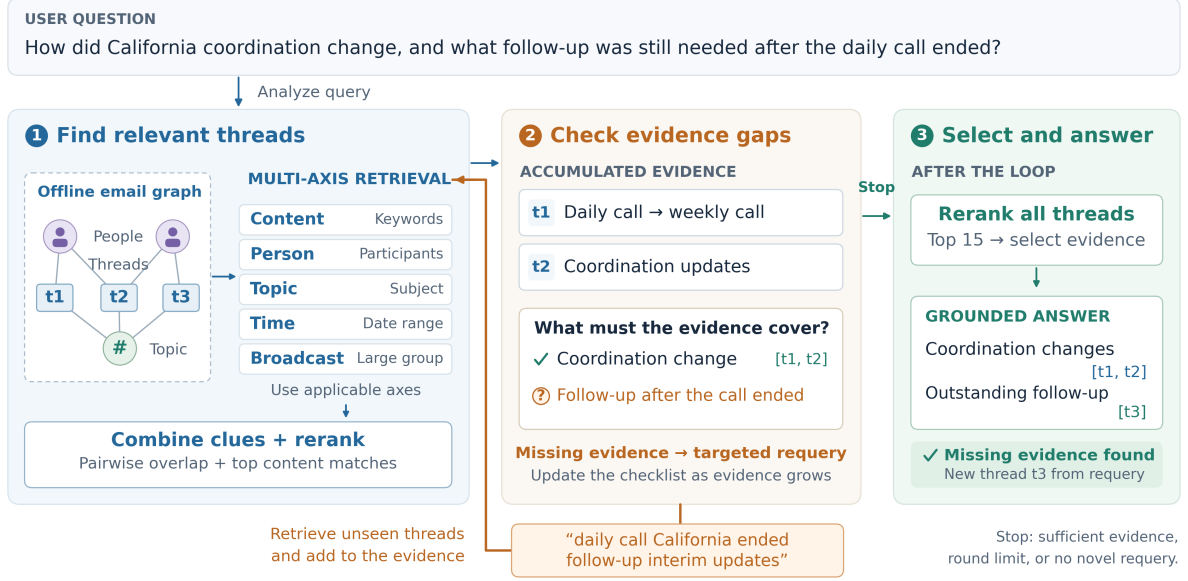


Figure 1: **Overview of PIKE.** An offline email graph supports retrieval over multiple signals relevant to emails, i.e. content, person, topic, time, and broadcast. An evidence-aware critic identifies information gaps and guides targeted retrieval of additional threads. After termination, PIKE reranks the accumulated evidence, retains the top 15 threads, and jointly selects evidence and generates an answer. The corresponding example and graph is [Is this example real? -RD].

events occurred, and which topics connect separate conversations. Building on email graphs and graph-augmented retrieval (Laclavík et al., 2012; Edge et al., 2024), PIKE combines these structural signals with content retrieval. An evidence-aware critic distinguishes partial coverage from sufficient evidence and guides query refinement when information appears to be missing. We further separate retrieval, evidence selection, and answer generation to examine whether recovered evidence is effectively used, rather than assuming that higher retrieval recall necessarily produces better answers. We make the following contributions.

- We formulate personalized email search as thread-level evidence retrieval and operationalize the task in a benchmark of 640 verified query–answer–evidence tuples for two public email datasets.
- We introduce PIKE, an email-specific framework that combines multiple signals with an evidence-aware critic to guide iterative retrieval under a fixed evidence budget.
- We compare PIKE with several retrieval baselines across several query categories, inbox sizes, retrieval strategies, evidence-selection methods, and LLM backends.

2 Task Formulation and Benchmarking

Email Thread Retrieval Task. We study personal inbox question answering. Given a user’s email inbox $\mathcal{E} = \{t_1, t_2, \dots, t_N\}$, where each t_i is an email thread, and a natural-language query q , a retrieval system must identify the subset of threads $\mathcal{T} \subseteq \mathcal{E}$ containing the evidence needed to answer q . A downstream language model then generates an answer conditioned on the retrieved threads. This formulation differs from general-purpose document retrieval in two ways. First, the retrieval unit is an email thread, which may contain either a single message or a multi-message conversation. Each thread can include messages with distinct senders, timestamps, and reply structure. Second, email threads carry rich structural metadata, including participants, dates, and CC fields, that is absent from standard text corpora but can be highly informative for retrieval.

QA Benchmark. We construct a benchmark of 640 question–answer pairs over 50 personal email collections: 25 Enron inboxes and 25 Avocado inboxes. As shown in Figure ??, the collections vary substantially in size, ranging from 20 to 10,615 threads per inbox. This variation enables evaluation across both sparse and content-rich

Category	Description and example
Keyword Lookup (KeyLook)	A keyword query answered by one email thread. <i>Example: Davis price caps</i>
Partial-Memory Lookup (PartMem)	An incomplete-memory query answered by one email thread. <i>Example: Which assistant sent the latest notice rescheduling the meeting with Kevin Presto?</i>
Event Progression (EventProg)	Tracks how an event develops across multiple email threads. <i>Example: What did Warren Prihoda send to Kevin Presto on May 2 at Don Black’s request, and what did Kevin Presto confirm later that day?</i>
Catch-Up Aggregation (Catchup)	Summarizes what happened or where things stand across multiple email threads. <i>Example: Catch me up on how the lunch plans with Kelly Ruprecht developed from 2001-06-21 through 2001-06-22.</i>
Action and Commitment Tracking (ActTrack)	Identifies follow-ups, owners, commitments, decisions, and status changes. <i>Example: What did Mark Stewart initially report and request regarding the unaccounted-for projector, and what later update and next step did Andrew Wagoner provide?</i>
Recurring Role Inference (RecRole)	Infers a recurring relationship or role from repeated interactions. <i>Example: How did Erik Cramer keep John Zufferli updated on the market?</i>

Table 1: Proposed benchmark of six question categories with descriptions and representative examples.

personal inboxes.

The benchmark captures variation along three dimensions: lexical specificity, the number of required source threads, and the temporal or relational structure connecting them. We organize the benchmark into six query types. These correspond to Keyword Lookup (KeyLook), Partial Memory Lookup (PartMem), Event Progression (EventProg), Catch-Up Aggregation (Catchup), Action and Commitment Tracking (ActTrack), and Recurring Role Inference (RecRole). We provide a description and corresponding example in Table 1

The construction process, illustrated in Figure 1 in Appendix A, first selects source threads using type-specific procedures. An LLM then generates a question and reference answer from the selected evidence. Every candidate undergoes a common evidence-audit and refinement process that verifies source identifiers, query-type requirements, and direct support for answer claims. The resulting records contain a question, reference answer, gold thread identifiers, and query-type label. Full generation and validation details are provided in Appendix A. To minimize evaluation bias, benchmark construction is independent of PIKE. The generation and auditing models operate only on the selected email threads and never interact with the retrieval pipeline, its structured axes, or its outputs.

3 PIKE Framework

We introduce PIKE (Personalized Inbox Knowledge-graph for Evidence-guided Retrieval), a framework for retrieving, and selecting evidence from personal email inboxes. PIKE is supported by

an offline email knowledge graph and consists of three online stages: (1) multi-axis initial retrieval, (2) critic-guided retrieval expansion, and (3) final evidence selection and answer generation. Figure 1 illustrates the complete pipeline. All system prompts are provided in Appendix ??.

3.1 Offline Email Graph Construction

PIKE constructs an email knowledge graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with three node types: person nodes $v_p \in \mathcal{V}_P$, topic nodes $v_t \in \mathcal{V}_T$, and thread nodes $v_c \in \mathcal{V}_C$. A thread may contain either one email or multiple messages from the same conversation; in both cases, it constitutes one retrieval unit. The person set $\mathcal{V}_P$ contains contacts who exchanged at least one message with the inbox owner. We define two direct edge types:

$$(v_p, v_c) \in \mathcal{E}_{\text{PART}} \quad \text{and} \quad (v_t, v_c) \in \mathcal{E}_{\text{ABOUT}},$$

where $\mathcal{E}_{\text{PART}}$ links a person to each thread in which they sent or received a message, and $\mathcal{E}_{\text{ABOUT}}$ links a topic to its associated threads. These edges serve as precomputed indexes, allowing person- and topic-based thread retrieval through a single graph hop. We additionally construct a person co-occurrence graph in which the weight between two people is the number of threads in which they both participate. Louvain community detection assigns each person node a community label $\gamma(v_p)$. For each community, we generate a summary of its members and representative thread subjects. These summaries provide optional context for queries concerning recurring participants, social groups, or cross-thread patterns.

3.2 Stage 1: Multi-Axis Retrieval

Query analysis. Given a user query q , an LLM query analyzer produces a keyword-oriented rewrite of the query (similar to query expansion in BM-25). It also identifies (i) the people referenced explicitly or implicitly, (ii) the relevant topics such as projects, events, or contracts, (iii) the time interval, and (iv) whether q is a broadcast message.

When available, community summaries are provided as additional context to the query analyzer for all query types. They are particularly useful for aggregation and cross-thread queries such as CatchUp, where the analyzer may expand the extracted person and topic signals using relevant community members and representative subjects.

Axis construction. The extracted signals define up to five retrieval axes $\mathcal{A}_1, \dots, \mathcal{A}_5$:

- **Content axis $\mathcal{A}_1$.** We retrieve the top 75 threads of the re-written query through reciprocal-rank fusion.
- **Person axis $\mathcal{A}_2$.** For every extracted person p_i , PIKE locates the corresponding person node using exact matching with partial-token fallback and retrieves its adjacent thread nodes:

$$\mathcal{A}_2 = \bigcup_i \{v_c \in \mathcal{N}(v_{p_i}) \mid v_c \in \mathcal{V}_C\}.$$

- **Topic axis $\mathcal{A}_3$.** For an extracted topic t_j , PIKE selects the topic node with most token overlap,

$$v_{t_j}^* = \arg \max_{v \in \mathcal{V}_T} |\text{tokens}(t_j) \cap \text{tokens}(v)|,$$

requiring at least two overlapping tokens, and retrieves its adjacent threads.

- **Time axis $\mathcal{A}_4$.** Threads whose timestamps overlap the inferred interval $[t_s, t_e]$. This axis is active only when a time range is available.
- **Broadcast axis $\mathcal{A}_5$.** Threads with at least ten participants. This axis is active only when `is_broadcast = true`.

Pairwise-intersection retrieval. A simple union of all axes can introduce many weakly related threads. PIKE instead constructs a pairwise-intersection pool:

$$\mathcal{P}_1 = \bigcup_{1 \leq i < j \leq 5} (\mathcal{A}_i \cap \mathcal{A}_j).$$

This pool favors threads supported by at least two independent signals, such as a thread matching both the requested person and time range.

To retain strong content matches when structured signals are missing or noisy, PIKE guarantees inclusion of the top 15 content results:

$$\mathcal{C}_1 = \mathcal{P}_1 \cup \text{Top}_{15}(\mathcal{A}_1).$$

A neural reranker scores each candidate thread against the original query q and returns the initial evidence set:

$$\mathcal{R}_1 = \text{Rerank}_{15}(q, \mathcal{C}_1).$$

3.3 Stage 2: Critic-Guided Retrieval Expansion

After Stage 1 produces the initial evidence set $\mathcal{R}_1$, PIKE uses a critic LLM to determine whether additional retrieval is needed. The accumulated evidence is initialized as $\mathcal{R} \leftarrow \mathcal{R}_1$, and PIKE permits up to $T = 4$ retrieval rounds in total. We formulate the critic decision as an evidence-checklist problem. At each critic step, the model determines which information is required, which requirements are covered by the current evidence, and which evidence remains missing. If the evidence is insufficient, the critic converts the missing information into a targeted keyword requery. We study three versions of critics that differ in when and how they construct their evidence requirements, as summarized in Table 5.

Static Checklist. The Static Checklist is generated once from the question before the critic examines any retrieved evidence. It contains one to three answer-bearing information needs:

$$\mathcal{E}^{\text{req}} = \text{BuildChecklist}(q).$$

At every round, the critic independently labels each checklist item as COVERED or NOT COVERED using the currently accumulated email threads. It returns

$$(\mathcal{E}_t^{\text{cov}}, \mathcal{E}_t^{\text{miss}}, s_t, q_{t+1}),$$

where $\mathcal{E}_t^{\text{cov}}$ and $\mathcal{E}_t^{\text{miss}}$ are the covered and uncovered checklist items, respectively, $s_t \in \{0, 1\}$ is the sufficiency decision, and q_{t+1} is a targeted requery. The evidence is declared sufficient only if every fixed checklist item is covered:

$$s_t = \mathbb{1}[\mathcal{E}_t^{\text{miss}} = \emptyset].$$

If any item remains uncovered, the critic generates q_{t+1} from the uncovered checklist items.

Dynamic Evidence Checklist. The Dynamic Evidence Checklist does not commit to a fixed requirement set before retrieval. Instead, it reconstructs the evidence requirements at every round using the question and the currently retrieved full-text email threads:

$$(\mathcal{E}_t^{\text{req}}, \mathcal{E}_t^{\text{cov}}, \mathcal{E}_t^{\text{miss}}, s_t, q_{t+1}) = \text{CheckEvidence}(q, \mathcal{R}).$$

Here, $\mathcal{E}_t^{\text{req}}$ is the dynamically inferred set of information requirements. The critic verifies whether every requirement is explicitly supported by the current evidence and sets

$$s_t = \mathbb{1}[\mathcal{E}_t^{\text{miss}} = \emptyset].$$

Because $\mathcal{E}_t^{\text{req}}$ is reconstructed after observing the evidence, it may change across rounds as newly retrieved threads resolve existing requirements or reveal additional information needed to answer the question. When evidence is missing, the critic produces a concise, BM25-oriented requery q_{t+1} targeting $\mathcal{E}_t^{\text{miss}}$.

Answer-Conditioned Dynamic Checklist. The Answer-Conditioned variant additionally attempts to generate a candidate answer y_t from the currently retrieved full-text threads. In the same LLM call, it identifies the information required by the question, evaluates whether the answer addresses that information, and verifies whether every factual claim in the answer is explicitly supported by the retrieved evidence:

$$(y_t, \mathcal{E}_t^{\text{req}}, \mathcal{E}_t^{\text{cov}}, \mathcal{E}_t^{\text{miss}}, s_t, q_{t+1}) = \text{DraftAndCheck}(q, \mathcal{R}).$$

The critic sets $s_t = 1$ only when the candidate answer is complete and all of its factual claims are grounded in the retrieved threads. Otherwise, it converts unsupported claims or missing information into a targeted requery q_{t+1} . The candidate answer itself is not propagated to the next round. Only the requery is passed to the retrieval pipeline; after new evidence is added, the answer is regenerated from the updated evidence.

Targeted multi-axis requery. When the critic returns insufficient, the generated requery q_{t+1} is passed through the same query analyzer used in Stage 1. The analyzer extracts new content, person, topic, temporal, broadcast, and community signals, and PIKE reconstructs the corresponding retrieval axes and pairwise-intersection pool. Let $\mathcal{C}_{t+1}$ denote the resulting candidate set.

To avoid repeatedly retrieving the same evidence, PIKE retains only previously unseen threads:

$$\Delta_{t+1} = \mathcal{C}_{t+1} \setminus \mathcal{R}.$$

The neural reranker scores Δ_{t+1} against the original user query q , rather than the critic-generated requery, and returns a new evidence batch:

$$\mathcal{B}_{t+1} = \text{Rerank}_k(q, \Delta_{t+1}).$$

The accumulated evidence is then updated as

$$\mathcal{R} \leftarrow \mathcal{R} \cup \mathcal{B}_{t+1}.$$

This procedure allows the requery to broaden candidate discovery while ensuring that newly added threads remain relevant to the original information need.

Termination. The expansion loop terminates under any of four conditions: (1) The critic declares the current evidence sufficient; (2) The critic returns no requery; (3) The generated requery duplicates a previous requery; or (4) The system reaches the maximum number of retrieval rounds. After termination, the accumulated set $\mathcal{R}$ is passed to Stage 3 for final reranking, evidence selection, and answer generation.

3.4 Stage 3: Reranking and Generation

The accumulated set $\mathcal{R}$ may contain more threads than can be used effectively for answer generation, particularly when the critic invokes multiple retrieval rounds. PIKE therefore reranks all accumulated threads against the original query and retains a fixed-size final candidate set:

$$\mathcal{S} = \text{Rerank}_{15}(q, \mathcal{R}).$$

Using a fixed top-15 set ensures that methods with more retrieval rounds do not receive a larger evidence budget. [We do this for other baselines as well to ensure fair comparison with PIKE. -RD] We provide the final list of 15 retrieved threads to an LLM for subsequent answer generation. We also explore different approaches of evidence selection but we defer them in the Appendix.

4 Experiments

[Need to state here the retrievers we are comparing with along with the different PIKE variants, and the corresponding LLM models and metrics for both retrieval and answer generation. -RD]

Baseline Retrievers:

LLM Backbones:

Metrics:

Category	Non-LLM Baselines				LLM Baselines		PIKE		
	BM25	DPR	BGE-M3	Hybrid	SIM-RAG	Multi-Axis	Static	Dynamic	Ans-Cond
KeyLook	<u>0.834</u>	0.241	0.772	0.772	0.841	0.717	0.772	0.821	0.814
PartMem	0.859	0.434	0.758	0.879	0.758	0.859	0.909	0.889	<u>0.899</u>
EventProg	0.913	0.424	0.843	0.903	0.748	0.871	0.956	<u>0.947</u>	<u>0.947</u>
Catchup	0.848	0.350	0.761	0.849	0.531	0.841	0.894	0.905	<u>0.901</u>
ActTrack	0.966	0.462	0.942	0.966	0.746	0.856	0.987	<u>0.979</u>	0.987
RecRole	0.606	0.285	0.601	0.637	0.450	0.588	0.716	<u>0.728</u>	0.732
Overall	0.806	0.336	0.745	0.802	0.647	0.764	0.841*	<u>0.853*</u>	0.853*

Table 2: GPT-5.2 Recall@15 by question category and retriever. Overall is the macro-average over all 640 questions. Best results are bold and second-best results are underlined. Rankings use the unrounded scores.

5 Results

5.1 Retrieval Performance

BM25 is a remarkably strong baseline for personal email search. Despite its simplicity, BM25 is the strongest single-pass baseline overall and performs particularly well when queries retain concrete lexical cues, such as names, topics, dates, and event-specific terms. This suggests that even indirect recollections in personal email often preserve enough term overlap for conventional lexical retrieval to remain highly effective. More structured retrieval is nevertheless complementary rather than uniformly superior: MultiAxis retrieval is more effective for some descriptively phrased and temporally distributed queries, while Hybrid retrieval is strongest for relationally diffuse questions. Overall, the results establish BM25 as a difficult baseline to surpass consistently and show that additional retrieval complexity must be justified by gains on query types where lexical matching alone is insufficient.

Dynamic evidence checklists enable efficient iterative retrieval. The Static Checklist defines evidence requirements before retrieval and continues searching until all requirements are satisfied. Because these requirements are not grounded in the retrieved threads, they can be overly specific or misaligned with the available evidence, contributing to a high false-negative rate and unnecessary retrieval rounds. In contrast, the Dynamic Evidence Checklist reconstructs the required, covered, and missing evidence at each round and formulates targeted requeries from the remaining gaps. It improves recall on Types A, D, and F while operating close to a single round on average. Although Static performs better on Types B, C, and E, it requires substantially more rounds. Overall, Dynamic provides a better recall–cost

tradeoff.

Answer conditioning detects more evidence gaps but does not meaningfully improve retrieval recall. Answer conditioning makes the critic more conservative by identifying additional perceived grounding and completeness gaps. As Table 9 shows, this behavior primarily increases false-negative sufficiency judgments, causing retrieval to continue after the available gold evidence has already been found. Across all query types, answer conditioning reduces false positives only modestly while more than doubling false negatives. The increase is especially pronounced for Type A and open-ended Type E queries, for which answer-level verification frequently triggers more retrieval. The Answer-Conditioned Checklist drafts an answer at each round and checks whether its claims and required information are explicitly supported by the retrieved threads. Although this procedure detects more perceived evidence gaps, the additional retrieval raises the average number of rounds without producing a meaningful improvement in final Recall@15 over the Dynamic Evidence Checklist.

PIKE’s advantage is more pronounced in larger inboxes. Figure ?? shows a more pronounced advantage for PIKE over BM25 in larger inboxes. Among the ten largest inboxes, containing 2,673–10,615 threads each, Dynamic PIKE achieves approximately 0.79 mean inbox-level Recall@15, compared with 0.71 for BM25, a gain of approximately 8 percentage points. Across all 50 inboxes, Dynamic PIKE also exhibits a weaker negative Spearman correlation between inbox size and Recall@15 ($\rho = -0.65$) than BM25 ($\rho = -0.78$). Thus, lower recall is less strongly

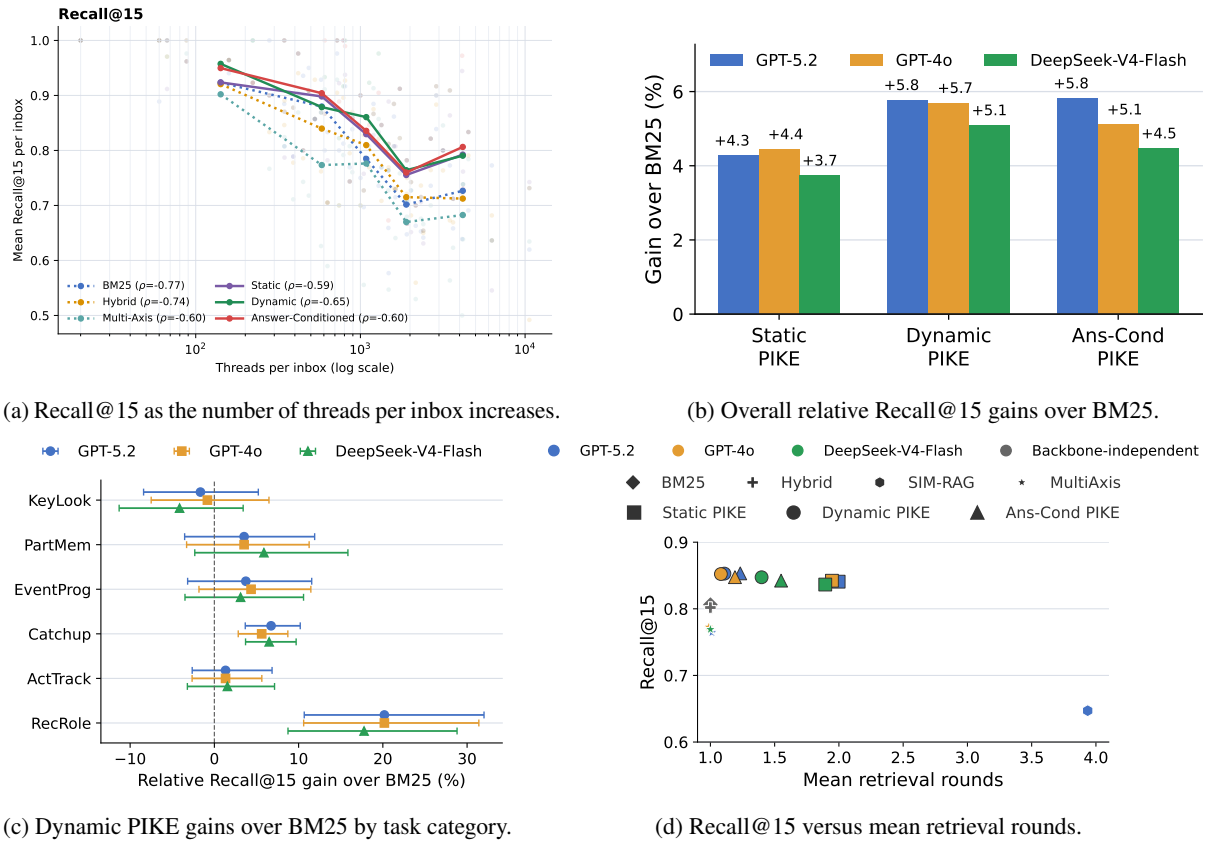


Figure 2: Retrieval effectiveness, efficiency, and robustness across PIKE variants and LLM backbones. The top row reports overall and task-specific gains over BM25; the bottom row shows the retrieval-round trade-off and sensitivity to inbox size. Colors identify LLM backbones where applicable, while marker shapes identify retrieval methods.

associated with larger inbox size for Dynamic PIKE.

Retrieval trends are consistent across LLM backends. We compare PIKE using GPT-5.2, GPT-4o (Hurst et al., 2024), and DeepSeek-V4-Flash (Xu et al., 2026) on the same benchmark with a fixed final Top-15 evidence budget. Figure ??(a) reports Dynamic PIKE’s mean Recall@15 difference from BM25 with paired 95% mailbox-cluster bootstrap confidence intervals (20,000 replicates). All three backends show negative point estimates on Type A and positive estimates on Types B–F, with the largest gains on F. The intervals exclude zero for D and F across all three backends, whereas those for A, B, C, and E include zero. Thus, the clearest gains over BM25 persist across the evaluated backends, although these comparisons do not establish equivalence between models. Figure ??(b) compares overall TermR@15 against mean retrieval rounds. Dynamic achieves the highest or near-highest recall in 1.09–1.40 rounds, compared with 1.89–1.99 for Static. Answer conditioning requires more rounds than Dynamic without

consistently improving recall. Full numerical results and additional variant comparisons appear in Appendix C.

5.2 PIKE Component Ablation

We examine PIKE’s structured retrieval contributions on 313 Type D and F questions using the Dynamic Evidence Checklist. Figure 3 reports paired changes in macro-averaged Recall@15 relative to Full PIKE, with 95% confidence intervals estimated using 20,000 mailbox-cluster bootstrap replicates. Removing all structured axes reduces recall by 6.3 percentage points on D and 13.6 points on F, with both confidence intervals excluding zero. Individual contributions differ across these types: removing the person axis reduces F by 9.7 points, while removing the time axis reduces D by 2.5 points. Topic removal produces little measurable incremental change for either type. These results indicate that structured retrieval supports both catch-up aggregation and recurring-role queries, with distinct contributions from temporal and person-based retrieval.

Category	Non-LLM Baselines				LLM Baselines		PIKE		
	BM25	DPR	BGE-M3	Hybrid	SIM-RAG	Multi-Axis	Static	Dynamic	Ans-Cond
KeyLook	0.836	0.299	0.777	0.794	0.841	0.757	0.775	<u>0.839</u>	0.822
PartMem	0.875	0.493	0.788	0.871	0.790	0.859	<u>0.891</u>	0.877	0.895
EventProg	0.918	0.618	0.914	0.936	0.832	0.918	0.959	0.927	<u>0.941</u>
Catchup	0.827	0.566	0.785	0.809	0.759	0.816	0.824	<u>0.826</u>	0.827
ActTrack	0.903	0.651	0.954	0.949	0.841	0.856	0.954	0.969	<u>0.959</u>
RecRole	0.780	0.552	0.787	<u>0.823</u>	0.720	0.761	0.820	0.812	0.838
Overall	0.837	0.500	0.803	0.836	0.784	0.807	0.840	<u>0.849</u>	0.855*

Table 3: LLM-as-Judge answer-similarity scores for GPT-5.2 with no post-retrieval selector, using all top-15 retrieved threads. Values are reported to three significant figures. Best results are bold and second-best results are underlined; rankings use unrounded scores. Overall is the macro-average over all 640 questions.

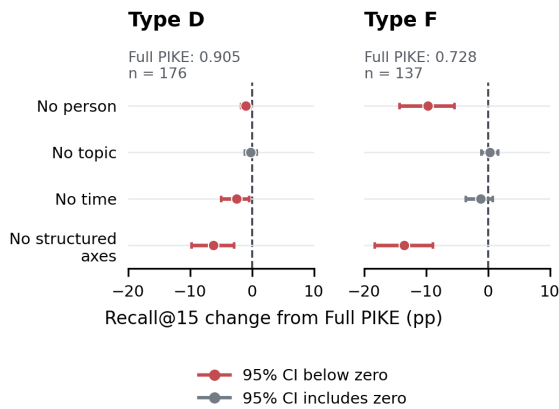


Figure 3: Structured retrieval ablation on Types D & F. Points show Recall@15 changes from Full PIKE in percentage points; error bars denote paired 95% mailbox-cluster bootstrap confidence intervals.

5.3 Analysis of Missing-Evidence Recovery

To identify where the Dynamic Evidence Checklist succeeds and where it loses relevant evidence, we decompose its retrieval process into three stages: expansion triggering, targeted requery, and final reranking. Figure 6 summarizes this analysis. Among the 640 evaluation questions, 153 (23.9%) were missing at least one annotated gold thread after Round 1. The critic triggered expansion for only 25 of these questions (16.3%) and terminated on the remaining 128 (83.7%). Once expansion was triggered, **targeted requery was highly effective**, recovering 23 of the 27 missing gold threads (85.2%). Final reranking, however, retained only 13 of these 23 recovered threads (56.5%) in the final Top-15. As a result, 12 of the 25 expanded questions (48.0%) were fully rescued. These results locate the main opportunities for improvement in deciding when to

expand and in retaining recovered evidence during final reranking, rather than in targeted requery itself.

5.4 Statistical Significance

A paired, question-level non-parametric bootstrap analysis with one million resamples showed that all three PIKE variants significantly improved Recall@15 over all baselines. Improvements in answer quality were less uniform when GPT-5.2 generated answers from all top-15 retrieved threads. Only ANS COND N PIKE significantly improved the LLM-as-Judge similarity score over all baselines, with other PIKE variants failing to significantly outperform BM-25 and the Hybrid baseline. These results suggest that PIKE consistently improves evidence retrieval, but those gains do not always translate into significantly better answers.

6 Related Works

[Needs to be in the later section after results since we already are comparing our approach with prior work. -RD]

Email Retrieval and Personal Knowledge Search. Despite email’s importance as a personal knowledge source, retrieval for question answering over individual inboxes remains comparatively underexplored. Earlier email research has addressed related tasks, including thread reconstruction (Yeh and Harnly, 2006), conversation summarization (Carenini et al., 2007), and graph-based relation discovery (Laclavík et al., 2012). Broader work has also examined search and discovery in personal email collections (Bendersky et al., 2022). These efforts provide useful foundations,

but address different tasks from retrieving the evidence needed to answer a user’s question across an entire inbox. EnronQA (Ryan et al., 2025) is particularly close to our setting: it provides approximately 528K single-hop questions grounded in individual emails, supporting research on personalized retrieval and factual memorization. QAConv (Wu et al., 2022) instead focuses on question answering over supplied conversational context, including email conversations. Our work extends this evaluation scope to six personal email search intents, including partial-memory retrieval and questions requiring evidence from multiple threads. The resulting benchmark contains 640 verified query–answer–evidence tuples from 50 Enron (Klimt and Yang, 2004) and Avocado (Oard et al., 2015) inboxes, enabling separate evaluation of retrieval, evidence selection, and answer generation.

Structured and Iterative Retrieval. Retrieval-augmented generation combines external retrieval with language models to support knowledge-grounded answers (Lewis et al., 2020; Izacard et al., 2023). GraphRAG (Edge et al., 2024) uses graph-derived community summaries to support query-focused summarization over document collections. PIKE instead uses an email graph as a structured index over people, topics, and threads, combining graph-based candidate discovery with content, temporal, and broadcast signals. This design targets evidence retrieval within a personal inbox rather than collection-level summarization. Other approaches adapt retrieval as information needs become clearer. Interleaved and active retrieval methods couple retrieval with intermediate reasoning or generation (Trivedi et al., 2023; Jiang et al., 2023), while RQ-RAG (Chan et al., 2024) learns query refinement through rewriting, decomposition, and disambiguation. PIKE likewise refines queries, but grounds refinement in explicit evidence-coverage judgments and executes the resulting queries through the same email-specific multi-axis retriever.

Evidence Assessment and Selection. SIM-RAG (Yang et al., 2025) trains an external critic through self-practicing to assess information sufficiency in multi-round RAG. Our Answer-Conditioned Checklist is inspired by its use of a tentative answer in sufficiency assessment. The critic drafts an answer, checks its completeness

and grounding against retrieved email threads, and formulates a requery when evidence appears insufficient. We compare this interface with static and dynamic evidence checklists that assess coverage without drafting an answer. All three are prompt-based and share the same retrieval mechanism, isolating the effect of how evidence requirements are constructed and evaluated. Self-RAG (Asai et al., 2023) trains a language model to emit reflection tokens for retrieval decisions, passage relevance, and generation support. Its explicit assessment of relevance and support motivates PIKE’s aspect-based selection-and-generation interface. Rather than reproducing Self-RAG’s learned reflection-token architecture, PIKE decomposes a question into information aspects and identifies supporting threads for each aspect through prompting. We compare this interface with flat selection and direct generation using identical retrieved candidates. This controlled comparison examines how evidence organization affects source precision, evidence coverage, and answer quality, without assuming that explicit selection uniformly improves generation.

7 Conclusion

Limitations

This document does not cover the content requirements for ACL or any other specific venue. Check the author instructions for information on maximum page lengths, the required “Limitations” section, and so on.

References

- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*.
- Michael Bendersky, Xuanhui Wang, Marc Najork, and Donald Metzler. 2022. Search and discovery in personal email collections. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*, pages 1617–1619.
- Giuseppe Carenini, Raymond T Ng, and Xiaodong Zhou. 2007. Summarizing email conversations with clue words. In *Proceedings of the 16th international conference on World Wide Web*, pages 91–100.
- Chi-Min Chan, Chunpu Xu, Ruibin Yuan, Hongyin Luo, Wei Xue, Yike Guo, and Jie Fu. 2024. Rq-rag: Learning to refine queries for retrieval augmented generation. *arXiv preprint arXiv:2404.00610*.

659	Darren Edge, Ha Trinh, Newman Cheng, Joshua	Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot,	714
660	Bradley, Alex Chao, Apurva Mody, Steven Truitt,	and Ashish Sabharwal. 2023. Interleaving retrieval	715
661	Dasha Metropolitansky, Robert Osazuwa Ness, and	with chain-of-thought reasoning for knowledge-	716
662	Jonathan Larson. 2024. From local to global: A	intensive multi-step questions. In <i>Proceedings of</i>	717
663	graph rag approach to query-focused summarization.	<i>the 61st annual meeting of the association for com-</i>	718
664	<i>arXiv preprint arXiv:2404.16130</i> .	<i>putational linguistics (volume 1: long papers)</i> , pages	719
		10014–10037.	720
665	Aaron Hurst, Adam Lerer, Adam P Goucher, Adam	Chien-Sheng Wu, Andrea Madotto, Wenhao Liu, Pas-	721
666	Perelman, Aditya Ramesh, Aidan Clark, AJ Os-	calle Fung, and Caiming Xiong. 2022. Qaconv: Ques-	722
667	trow, Akila Welihinda, Alan Hayes, Alec Radford,	tion answering on informative conversations. In <i>Pro-</i>	723
668	et al. 2024. Gpt-4o system card. <i>arXiv preprint</i>	<i>ceedings of the 60th Annual Meeting of the Associa-</i>	724
669	<i>arXiv:2410.21276</i> .	<i>tion for Computational Linguistics (Volume 1: Long</i>	725
		<i>Papers)</i> , pages 5389–5411.	726
670	Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas	Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang,	727
671	Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-	Bingzheng Xu, Bochao Wu, Bowei Zhang, Chao-	728
672	Yu, Armand Joulin, Sebastian Riedel, and Edouard	fan Lin, Chen Dong, Chenchen Ling, et al.	729
673	Grave. 2023. Atlas: Few-shot learning with retrieval	2026. Deepseek-v4: Towards highly efficient	730
674	augmented language models. <i>Journal of Machine</i>	million-token context intelligence. <i>arXiv preprint</i>	731
675	<i>Learning Research</i> , 24(251):1–43.	<i>arXiv:2606.19348</i> .	732
676	Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun,	Diji Yang, Linda Zeng, Jinmeng Rao, and Yi Zhang.	733
677	Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie	2025. Knowing you don’t know: Learning when	734
678	Callan, and Graham Neubig. 2023. Active retrieval	to continue search in multi-round rag through self-	735
679	augmented generation. In <i>Proceedings of the 2023</i>	practicing. In <i>Proceedings of the 48th International</i>	736
680	<i>conference on empirical methods in natural language</i>	<i>ACM SIGIR Conference on Research and Develop-</i>	737
681	<i>processing</i> , pages 7969–7992.	<i>ment in Information Retrieval</i> , pages 1305–1315.	738
682	Bryan Klimt and Yiming Yang. 2004. The Enron corpus:	Jen-Yuan Yeh and Aaron Harnly. 2006. Email thread	739
683	A new dataset for email classification research . In	reassembly using similarity matching.	740
684	<i>Machine Learning: ECML 2004</i> , pages 217–226.		
685	Springer.		
686	Michal Laclavík, Štefan Dlugolinský, Martin Šeleng,		
687	Marek Ciglan, and Ladislav Hluchý. 2012. Emails		
688	as graph: relation discovery in email archive. In		
689	<i>Proceedings of the 21st International Conference on</i>		
690	<i>World Wide Web</i> , pages 841–846.		
691	Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio		
692	Petroni, Vladimir Karpukhin, Naman Goyal, Hein-		
693	rich Küttler, Mike Lewis, Wen-tau Yih, Tim Rock-		
694	täschel, et al. 2020. Retrieval-augmented generation		
695	for knowledge-intensive nlp tasks. <i>Advances in neu-</i>		
696	<i>ral information processing systems</i> , 33:9459–9474.		
697	Fengran Mo, Kelong Mao, Ziliang Zhao, Hongjin Qian,		
698	Haonan Chen, Yiruo Cheng, Xiaoxi Li, Yutao Zhu,		
699	Zhicheng Dou, and Jian-Yun Nie. 2025. A survey of		
700	conversational search. <i>ACM Transactions on Inform-</i>		
701	<i>ation Systems</i> , 43(6):1–50.		
702	Douglas Oard, William Webber, David A. Kirsch,		
703	and Sergey Golitsynskiy. 2015. Avocado research		
704	email collection . Linguistic Data Consortium.		
705	LDC2015T03.		
706	Stephen Robertson and Hugo Zaragoza. 2009. The		
707	probabilistic relevance framework: BM25 and be-		
708	yond . <i>Foundations and Trends in Information Re-</i>		
709	<i>trieval</i> , 3(4):333–389.		
710	Michael J Ryan, Danmei Xu, Chris Nivera, and		
711	Daniel Campos. 2025. Enronqa: Towards person-		
712	alized rag over private documents. <i>arXiv preprint</i>		
713	<i>arXiv:2505.00263</i> .		

A Email Search QA Benchmark Construction

We construct an evidence-grounded question answering benchmark representing six information needs commonly encountered in personal email search. The benchmark contains 640 questions from 50 personal inboxes, comprising 25 Enron users and 25 Avocado users. Each question is paired with a reference answer and a non-empty set of gold email threads.

The benchmark covers explicit keyword lookup (Type A), retrieval from partial memory (Type B), cross-thread event progression (Type C), open-ended catch-up aggregation (Type D), action and commitment tracking (Type E), and recurring relationship or role inference (Type F). These categories represent distinct search intents rather than a strict ordering of difficulty.

- **Type A – Keyword lookup.** Short keyword fragments containing explicit person, topic, or detail cues. Each question is grounded in a single source thread.
- **Type B – Partial memory.** Natural-language recollections that omit exact identifiers such as names, dates, subject lines, or distinctive phrases. Each question is grounded in a single thread, isolating retrieval under lexical mismatch.
- **Type C – Event progression.** Questions asking how a concrete event or workflow evolved across multiple threads, such as a progression from proposal to revision and decision. The gold evidence contains the distinct threads required to reconstruct the event stages.
- **Type D – Catch-up.** Open-ended requests to summarize the state of a project, issue, or relationship over a specified period. Answers aggregate source-supported updates, decisions, and outcomes distributed across multiple threads.
- **Type E – Action tracking.** Questions asking which actions, commitments, or next steps emerged from a sequence of project updates. Gold threads are the union of the sources supporting the required action-relevant points.
- **Type F – Recurring roles.** Questions about a person’s recurring role or pattern of involvement across separate interactions. Answers require evidence from multiple threads and exclude coincidental co-occurrence without repeated interaction.

Let $\mathcal{I}_u$ denote the conversation threads in user u ’s inbox. Each benchmark instance is

$$z_q = (x_q, y_q, G_q, \tau_q), \quad (1)$$

where x_q is the query, y_q is the reference answer, $G_q \subseteq \mathcal{I}_u$ is the gold thread set, and $\tau_q \in \{A, B, C, D, E, F\}$ is the query type. Types A and B have exactly one gold thread, whereas Types C–F require at least two.

We use type-specific construction procedures rather than a uniform generation template. Single-source questions are generated from preselected threads, while multi-source questions are constructed from event clusters, source-linked answer points, or recurring-interaction groups. Candidate generation is followed by a common evidence audit and refinement stage described in Section A.6.

A.1 Types A and B: Source-First Retrieval Queries

Types A and B use a source-first procedure. We first select an eligible thread and assign it a fixed query subtype. Eligible threads contain at least one message and 80 body characters. We exclude threads with more than 20 non-owner participants or serialized representations longer than 60,000 characters, removing empty messages, broad mass-distribution email, and exceptionally large conversations.

For each inbox, we attempt to generate three Type A and two Type B questions from five distinct threads:

$$N_{AB}^{(u)} \leq 3_A + 2_B. \quad (2)$$

The Type A subtypes are person–topic, topic-only, and specific-detail queries. Type B contains one “who was it” and one “what happened” query. A selected source thread is not reused for another A/B question in the same inbox.

The generation model receives the complete thread, including its subject, participants, timestamps, and message bodies. The expected subtype and source identifier are fixed before generation. We reject outputs whose type, subtype, or returned source does not match this specification.

Type A: explicit keyword retrieval. A Type A query is a two-to-five-word keyword fragment rather than a complete question. It represents a user who remembers explicit search terms, such as a person and topic, a topic phrase, or a specific detail such as a phone number, passcode, monetary value, or percentage.

Type	Target capability	Questions	Gold threads
A	Explicit keyword retrieval	145	1
B	Partial-memory retrieval	99	1
C	Cross-thread event progression	44	2–4
D	Open-ended catch-up aggregation	176	2–10
E	Action and commitment tracking	39	2–7
F	Recurring relationship and role inference	137	2–3
Total		640	

Table 4: Composition of the 50-inbox email-search benchmark. Gold threads denote the observed range of gold evidence-set sizes.

Type B: partial-memory retrieval. A Type B query represents a user who remembers a situation but not its exact identifying details. Exact names, dates, numbers, subject lines, and direct quotations are omitted. Instead, the question retains two or three approximate semantic anchors, such as a role, event, document, request, or outcome. This makes the intended source identifiable by meaning without reducing the task to exact-string matching.

For both types, the selected thread defines the gold evidence set:

$$G_q = \{t_q\}, \quad \tau_q \in \{A, B\}. \quad (3)$$

Account handles and aliases are replaced with human-readable names only when supported by the source. Intentionally omitted Type B entities are not reintroduced. We also reject duplicate questions and outputs that violate subtype-specific structural constraints.

A.2 Type C: Cross-Thread Event Progression

Type C evaluates whether a system can reconstruct an event whose stages are distributed across separate threads. Candidate events are identified using semantic, subject, participant, and temporal signals.

Messages are divided into bounded chunks and embedded using all-MiniLM-L6-v2. The embedding of thread i is the normalized mean of its K_i message-chunk embeddings:

$$\mathbf{e}_i = \text{normalize} \left(\frac{1}{K_i} \sum_{k=1}^{K_i} \mathbf{e}_{i,k} \right). \quad (4)$$

For each thread, we retain up to 30 semantic neighbors. We also introduce candidate pairs sharing a normalized subject or participant. Pair affinity is

$$S(i, j) = 0.55 S_{\text{sem}}(i, j) + 0.20 S_{\text{subj}}(i, j) + 0.15 S_{\text{part}}(i, j) + 0.10 S_{\text{time}}(i, j) \quad (5)$$

where semantic similarity is cosine similarity, subject and participant similarities are Jaccard similarities, and

$$S_{\text{time}}(i, j) = 2^{-\Delta d_{ij}/90}. \quad (6)$$

Pairs satisfying $S(i, j) \geq 0.45$ form weighted edges.

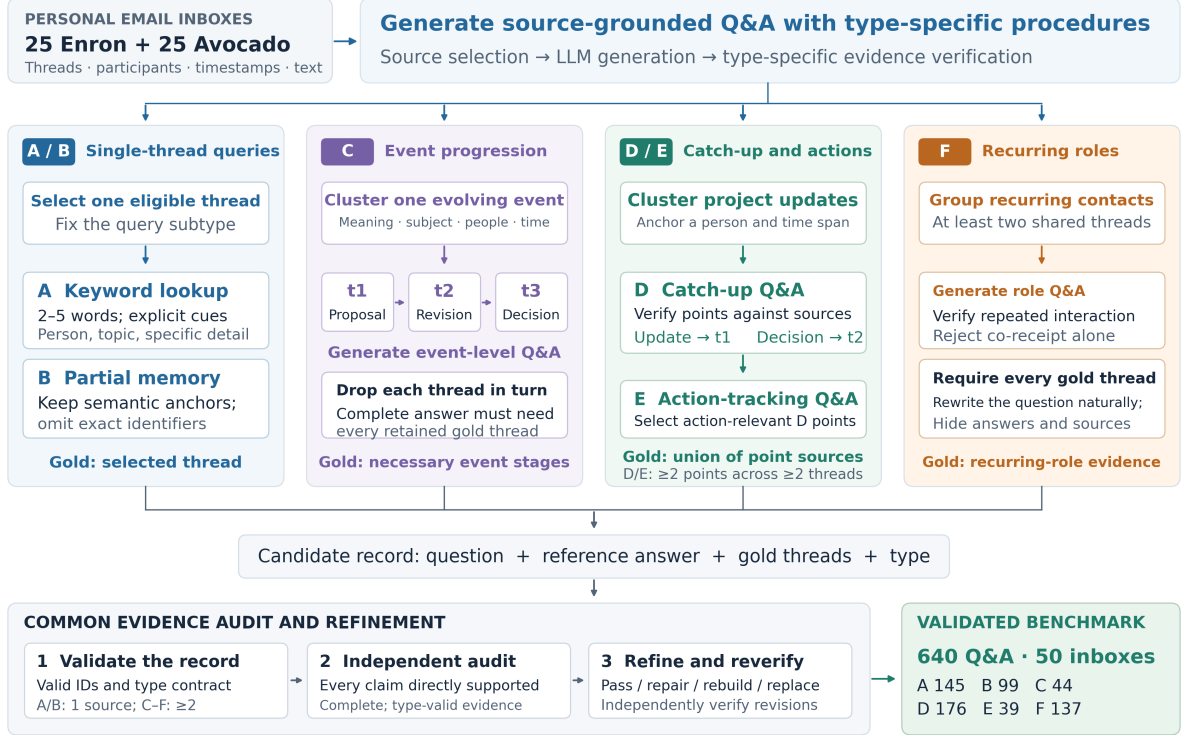
Clusters are initialized from high-weighted-degree threads. A candidate c is added to cluster C when its mean affinity

$$A(c, C) = \frac{1}{|C|} \sum_{t \in C} S(c, t) \quad (7)$$

is at least 0.30. We construct at most 30 clusters per inbox, each containing at most eight threads. We remove clusters spanning more than 60 days and near-duplicates with thread-set Jaccard overlap of at least 0.8.

An LLM adjudicator retains a cluster only if it represents a coherent event with meaningful progression rather than repeated templates. Specifically, the cluster must contain at least two stages supported by distinct threads, such as *proposal*, *revision*, and *decision* or *problem*, *response*, and *resolution*. From each accepted cluster, the adjudicator selects two to five threads required to reconstruct this progression. We then rank the resulting candidates by continuity score, retain only those scoring at least 0.70, and remove lower-ranked candidates that share a selected source with a higher-ranked candidate. Finally, we retain at most three event questions per inbox.

Questions are normalized to use evidence-supported names and to remove unsupported causal



Generated independently of PIKE retrieval; generation and audit metadata are held out from the retrieval system.

Figure 4: **Benchmark construction pipeline.** Source-grounded question–answer pairs are generated from 25 Enron and 25 Avocado inboxes using type-specific source-selection and verification procedures. All candidates then undergo a common evidence audit and refinement process, yielding 640 validated tuples of a question, reference answer, and gold email threads across Types A–F. Benchmark construction is independent of PIKE retrieval.

wording. The complete gold set must support every substantive answer claim. We then test each gold thread by removing it and checking the remaining evidence. A candidate is retained only when the full set supports the answer and removing any one thread makes the remaining evidence insufficient to support the complete answer.

A.3 Type D: Open-Ended Catch-Up Aggregation

Type D models requests for an update on a concrete project, deal, deliverable, or issue involving a particular person. Unlike Type C, which reconstructs the ordered stages of one event, Type D may aggregate multiple updates, decisions, changes, and unresolved issues. It consequently has the largest evidence sets, with a mean of 5.22 and a maximum of ten gold threads.

We first embed each email thread using all-MiniLM-L6-v2. Messages are divided into bounded chunks when necessary, and each thread is represented by the normalized mean of its message-

chunk embeddings:

$$\mathbf{e}_i = \text{normalize} \left(\frac{1}{K_i} \sum_{k=1}^{K_i} \mathbf{e}_{i,k} \right), \quad (8)$$

where K_i is the number of embedded chunks in thread i .

We then apply agglomerative clustering using cosine distance and average linkage. Agglomerative clustering begins with each email thread as a singleton cluster. At each step, it merges the two clusters with the smallest average cross-thread cosine distance, provided that this distance does not exceed 0.35. Merging stops when no pair of clusters satisfies this threshold. We then retain clusters containing between three and ten threads.

Smaller clusters with less than 3 threads provide insufficient material for a multi-thread catch-up question, whereas larger clusters more than 10 threads are more likely to combine multiple unrelated projects. For each retained cluster, we select as the person anchor the non-owner participant who appears in the largest number of its threads. We also derive an explicit time span from the earli-

est and latest thread dates. Up to four candidate clusters are examined per inbox.

Embedding similarity alone does not guarantee that a cluster concerns one project. An LLM therefore reads the candidate threads and verifies that they describe one concrete project, deliverable, deal, or issue involving the selected person. It rejects clusters that share only a broad topic, contain unrelated activities, or lack enough substantive updates for a catch-up answer.

For each valid cluster, the model generates a natural catch-up query, a synthesized reference answer, and a source-linked key-point rubric:

$$R_q = \{(r_j, S_j)\}_{j=1}^m, \quad (9)$$

where r_j is a substantive answer point and $S_j \subseteq \mathcal{I}_u$ is the set of threads that supports it. For example, one point may describe a requested revision and cite one thread, while another describes a pending approval and cites a different thread.

A separate evidence verifier reads the complete candidate threads and checks that every answer point is directly supported by its cited source IDs. It also rejects unsupported claims about participation, causality, completion, or outcomes. A candidate is retained only if it contains at least two substantive answer points whose supporting sources span at least two distinct threads. The gold evidence set is then computed as

$$G_q = \bigcup_{j=1}^m S_j. \quad (10)$$

Deriving the gold set from verified point-level provenance prevents unrelated or nonexistent conversation identifiers from entering the annotation. The common audit described in Section A.6 subsequently rechecks every answer claim and verifies that the question defines a coherent catch-up target rather than merely enumerating unrelated notifications.

A.4 Type E: Action and Commitment Tracking

Type E targets concrete actions and their states across multiple threads. A question may ask who owns an action, what follow-up was requested, when an item is due, whether it remains pending, or what should happen next. General catch-up questions are excluded.

Type E candidates are derived from the source-linked answer points constructed for Type D. Each

Type D rubric contains a set of substantive facts, with every fact explicitly linked to the email threads that support it. We rank these point sets using action-related signals such as ownership, approval, deadlines, requests, follow-up, completion, and pending status. Up to three high-scoring Type D point sets are considered for each inbox, and at most two Type E questions are generated.

The generation model receives the available evidence points and their fixed point identifiers. It cannot directly propose gold thread identifiers. Instead, it must select at least two supplied points:

$$R_q^* \subseteq R_q, \quad |R_q^*| \geq 2. \quad (11)$$

The selected points must collectively span at least two distinct threads. The gold evidence set is computed in code from the sources attached to those points:

$$G_q = \bigcup_{(r_j, S_j) \in R_q^*} S_j. \quad (12)$$

This design prevents the generation model from inventing or misattributing gold conversation identifiers.

A separate verifier checks that the question is fully answerable from the selected points and that the reference answer makes no stronger claim than those points support. In particular, a request, suggestion, or planned action is not treated as evidence that the action was completed. When necessary, the verifier may conservatively revise the question and answer or select different supplied points from the same Type D rubric. A candidate is discarded if it cannot be made fully supported while retaining at least two evidence points across at least two threads.

A.5 Type F: Recurring Relationship and Role Inference

Type F evaluates whether separate interactions jointly reveal a recurring working relationship or operational role, such as repeated information sharing, coordination, reporting, approval, handoffs, or collaboration.

Let P_t be the non-owner participants in thread t . For participant p , define

$$\mathcal{T}(p) = \{t \in \mathcal{I}_u : p \in P_t\}. \quad (13)$$

For a participant pair (p_i, p_j) , the thread-level co-occurrence count is

$$f(p_i, p_j) = |\mathcal{T}(p_i) \cap \mathcal{T}(p_j)|. \quad (14)$$

We consider pairs with $f(p_i, p_j) \geq 2$, rank them by co-occurrence frequency, and retain up to ten threads per candidate group. Participants are not reused across selected groups, and up to ten groups are examined per inbox.

The generation model must produce a question involving the participants, a concrete activity domain, and a recurring task, responsibility, decision, document, or operational role. Generic questions such as “What was their relationship?” are disallowed.

Co-occurrence alone is not sufficient. A relationship verifier requires the same substantive role to be supported across multiple threads through direct interaction, repeated work, handoffs, reporting, shared decisions, or explicit collaboration. Mere co-receipt, distribution-list membership, and unrelated broadcasts are rejected.

The full gold set must support the answer. We additionally remove each thread in turn and reassess the remaining evidence. A candidate is retained only when every gold thread contributes necessary evidence to the recurring role or relationship. A separate verifier confirms that the inferred role is consistent across the retained threads and is not based only on participant overlap.

Finally, questions are rewritten as natural requests from an inbox owner. The rewriter sees only the question, not the answer or source threads, preventing new evidence terms from becoming retrieval cues. A verifier retains the rewrite only when it preserves the intended meaning and removes benchmark-oriented expressions such as “recurring relationship” or “across the emails.”

A.6 Canonical Audit and Refinement

After type-specific construction, we assemble 640 question slots and conduct a common item-level audit. For every item, an independent evaluator receives the question, reference answer, type contract, and complete gold-thread contents.

Deterministic checks require non-empty questions and answers, valid type labels, existing conversation identifiers, exactly one gold thread for Types A and B, and at least two for Types C–F. Type B additionally must be a grammatical question rather than a malformed keyword fragment.

The evidence audit checks that every substantive answer claim is directly supported, the answer fully addresses the question, the question is natural and coherent, and the item satisfies its type contract. Multi-thread items must genuinely require

information from at least two threads. The Type D audit also rejects malformed temporal scopes and notification lists lacking a meaningful catch-up target. The Type F audit requires repeated interaction or responsibility beyond co-receipt or broadcast overlap.

Each item receives one of four actions: PASS, REPAIR-TEXT, REBUILD-ITEM, or REPLACE-GOLD. Text repair preserves the original intent; rebuilding creates a new same-type item from the same sources; and gold replacement is used when those sources cannot support a valid item of the assigned type.

Of the 640 items, 347 passed unchanged, 254 required text repair, 17 required same-source rebuilding, and 22 required source replacement. A second independent verifier reviewed all 271 proposed repairs and rebuilds, accepting 267 and routing four additional items to source replacement.

The resulting 26 unresolved slots were filled through targeted same-type generation. Each replacement was checked against its complete source threads for grounding, answer completeness, question naturalness, type validity, and multi-thread dependence. No question identifier was removed: the final set contains 347 unchanged items, 267 independently verified repairs or same-source rebuilds, and 26 independently verified source replacements.

A.7 Final Assembly and Release Validation

The final release contains 640 unique question identifiers from 50 users with the distribution in Table 4. No-evidence questions are excluded, so every item has a non-empty gold set.

Release validation rejects duplicate normalized questions and conversation identifiers absent from the corresponding inbox. It enforces exactly one gold source for Types A and B and at least two for Types C–F. It also verifies that all accepted refinements and 26 source replacements are applied exactly once without changing question types.

The released files contain only the fields required for evaluation:

```
{
  "id": "...",
  "question": "...",
  "answer": "...",
  "gold_source_conv_ids": ["conv_...", "..."],
  "difficulty": "A|B|C|D|E|F"
}
```

Generation metadata, source-linked points, audit

Table 5: Critic interfaces for retrieval expansion. All variants share the retrieval procedure in Algorithm 1 but differ in how they construct and evaluate evidence requirements.

Critic	Requirement construction	Sufficiency check
Static Checklist	Once, from the question	Fixed item coverage
Dynamic Evidence Checklist	Every round, from question and evidence	Dynamic evidence coverage
Answer-Conditioned Checklist	Every round, after drafting an answer	Answer completeness and grounding

Algorithm 1 Critic-guided retrieval expansion

Require: Question q ; initial evidence $\mathcal{R}_1$; maximum rounds T ; critic variant v
Ensure: Accumulated evidence $\mathcal{R}$

```

1:  $\mathcal{R} \leftarrow \mathcal{R}_1$ 
2:  $\mathcal{Q} \leftarrow \emptyset$  ▷ Previously issued requeries
3: if  $v = \text{STATIC}$  then
4:    $\mathcal{E}^{\text{req}} \leftarrow \text{BUILD\_CHECKLIST}(q)$ 
5: end if
6: for  $t = 1, \dots, T$  do
7:   if  $v = \text{STATIC}$  then
8:      $(\mathcal{E}_t^{\text{cov}}, \mathcal{E}_t^{\text{miss}}, s_t, q_{t+1}) \leftarrow \text{CHECKFIXED}(q, \mathcal{R}, \mathcal{E}^{\text{req}})$ 
9:   else if  $v = \text{DYNAMIC\_EVIDENCE}$  then
10:     $(\mathcal{E}_t^{\text{req}}, \mathcal{E}_t^{\text{cov}}, \mathcal{E}_t^{\text{miss}}, s_t, q_{t+1}) \leftarrow \text{CHECKEVIDENCE}(q, \mathcal{R})$ 
11:   else if  $v = \text{ANSWER\_CONDITIONED}$  then
12:     $(y_t, \mathcal{E}_t^{\text{req}}, \mathcal{E}_t^{\text{cov}}, \mathcal{E}_t^{\text{miss}}, s_t, q_{t+1}) \leftarrow \text{DRAFT\_AND\_CHECK}(q, \mathcal{R})$ 
13:   end if
14:   if  $s_t = 1$  then
15:     break ▷ Evidence is sufficient
16:   end if
17:   if  $t = T$  then
18:     break ▷ Maximum retrieval budget reached
19:   end if
20:   if  $q_{t+1} = \emptyset$  or  $q_{t+1} \in \mathcal{Q}$  then
21:     break ▷ No novel requery
22:   end if
23:    $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{q_{t+1}\}$ 
24:    $\mathcal{C}_{t+1} \leftarrow \text{MULTI\_AXIS\_RETRIEVE}(q_{t+1})$ 
25:    $\Delta_{t+1} \leftarrow \mathcal{C}_{t+1} \setminus \mathcal{R}$ 
26:    $\mathcal{B}_{t+1} \leftarrow \text{RERANK}(q, \Delta_{t+1})$ 
27:    $\mathcal{R} \leftarrow \mathcal{R} \cup \mathcal{B}_{t+1}$ 
28: end for
29: return  $\mathcal{R}$ 

```

decisions, evidence assessments, and replacement provenance are stored separately and are not provided to the retrieval system. A release manifest records the user count, type distribution, number of verified replacements, and file-level SHA-256 hashes for the exact benchmark snapshot used in our experiments.

B Additional Experimental Details

B.1 Algorithm for PIKE implementation

B.2 Evidence Based Selection

PIKE then jointly selects evidence from $\mathcal{S}$ and generates the final answer. We study two single-call selection-and-generation interfaces.

Flat selection and generation. The Flat+Gen interface directly selects the subset of threads in $\mathcal{S}$ that are necessary to answer the question and generates an answer using only those threads:

$$(\hat{\mathcal{S}}_{\text{flat}}, y_{\text{flat}}) = f_{\text{flat}}(q, \mathcal{S}).$$

The prompt explicitly instructs the model to exclude tangential or redundant threads, encouraging a compact and high-precision evidence set.

Aspect-based selection and generation. The Aspect+Gen interface first decomposes the information need into a set of question-specific aspects:

$$\mathcal{Z} = \{z_1, \dots, z_m\}.$$

For each aspect z_i , the model identifies its supporting thread set $\mathcal{S}_i$ and predicts a support label:

$$h_i \in \{\text{FULL}, \text{PARTIAL}, \text{NONE}\}.$$

The selected evidence is the union of citations associated with all supported aspects:

$$\hat{\mathcal{S}}_{\text{aspect}} = \bigcup_{i: h_i \neq \text{NONE}} \mathcal{S}_i.$$

In the same call, the model generates an answer covering every supported aspect using only the associated evidence:

$$y_{\text{aspect}} = f_{\text{aspect}}(q, \mathcal{S}, \mathcal{Z}).$$

Flat+Gen favors conservative evidence selection and higher source precision, whereas Aspect+Gen favors broader source recall and required-information coverage. Both interfaces receive exactly the same final top-15 candidate threads, allowing us to isolate how evidence organization affects source selection and downstream answer completeness.

C Additional Results

We evaluate whether PIKE’s retrieval behavior is robust to the choice of LLM backend. All experiments use the same 640 questions, top-15 retrieval budget, reranker, and evaluation protocol. BM25 and Hybrid are model-independent and are therefore shared across runs.

Table 6: Type-wise TermR@15 across LLM backends and PIKE variants. Rounds denotes the mean number of retrieval rounds.

Backend	Method	Overall	A	B	C	D	E	F	Rounds
Shared	BM25	0.806	0.834	0.859	0.913	0.848	0.966	0.606	1.00
Shared	Hybrid	0.802	0.772	0.879	0.903	0.849	0.966	0.637	1.00
GPT-5.2	MultiAxis	0.764	0.717	0.859	0.871	0.841	0.856	0.588	1.00
GPT-5.2	Static	0.841	0.772	0.909	0.956	0.894	0.987	0.716	1.99
GPT-5.2	Dynamic	0.853	0.821	0.889	0.947	0.905	0.979	0.728	1.10
GPT-5.2	Answer-Cond.	0.853	0.814	0.899	0.947	0.901	0.987	0.732	1.22
GPT-4o	MultiAxis	0.773	0.717	0.818	0.900	0.866	0.864	0.612	1.00
GPT-4o	Static	0.842	0.772	0.899	0.949	0.894	0.987	0.733	1.96
GPT-4o	Dynamic	0.852	0.828	0.889	0.953	0.895	0.979	0.728	1.09
GPT-4o	Answer-Cond.	0.848	0.814	0.889	0.953	0.899	0.987	0.714	1.20
DeepSeek-V4-Flash	MultiAxis	0.769	0.738	0.788	0.892	0.863	0.864	0.601	1.00
DeepSeek-V4-Flash	Static	0.836	0.793	0.859	0.945	0.896	0.987	0.712	1.89
DeepSeek-V4-Flash	Dynamic	0.847	0.800	0.909	0.941	0.903	0.981	0.714	1.40
DeepSeek-V4-Flash	Answer-Cond.	0.842	0.793	0.889	0.941	0.894	0.987	0.721	1.55

Type-level robustness. The three LLM backends produce broadly similar performance patterns across query types. In particular, critic-guided retrieval consistently yields its largest improvement over BM25 on Type F, while Type A remains favorable to lexical retrieval. This agreement indicates that the observed variation is primarily associated with query type and retrieval strategy rather than with a particular LLM backend.

Model-level differences. GPT-4o yields slightly higher point estimates than DeepSeek-V4-Flash across all four LLM-dependent variants. However, paired mailbox-cluster bootstrap confidence intervals include zero in every comparison. The models therefore exhibit comparable retrieval effectiveness, although GPT-4o terminates Dynamic retrieval more efficiently.

Audit of Critic Stopping Decisions. To assess whether the Dynamic critic stops despite missing required information, we independently audit all 640 saved Round 1 retrieval states. GPT-5.6-sol receives each question, its retrieved Top-15 threads, and a frozen rubric of required answer points. It labels each point as fully supported, partially supported, or unsupported; a state is classified as sufficient only when all points are fully supported. The judge does not see the original critic verdict or which threads are annotated as gold. The critic agrees with the judge on 82.8% of questions, but this aggregate conceals a strong class imbalance: the judge considers 531 of 640 states (83.0%) sufficient, so always predicting *sufficient* would achieve 83.0% agreement. Although the critic correctly accepts 508 of these 531 states (95.7%), it identifies only 22 of the 109 judge-insufficient states (20.2%) as insufficient. It labels the remaining

87 (79.8%) as sufficient, yielding a balanced accuracy of 57.9%. Thus, high overall agreement does not establish reliable gap detection: relative to the rubric-based judge, the critic frequently overlooks residual evidence gaps.

C.1 Selection & Generation

Flat selection favors precision, whereas aspect-based selection better preserves evidence coverage. Let G_q denote the gold thread set for question q , and let S_q denote the threads retained by the selector from the final top-15 candidates. We measure selection recall and precision as

$$\text{Rec}(q) = \frac{|S_q \cap G_q|}{|G_q|}, \quad \text{Prec}(q) = \frac{|S_q \cap G_q|}{|S_q|}.$$

The reported scores are macro-averaged across questions. Table 7 shows that aspect-based selection achieves higher recall for every query type, while flat selection consistently achieves higher precision. Flat selection consequently yields higher F1 for Types A, B, C, and F, as well as marginally higher F1 overall. In contrast, the recall gain from aspect decomposition outweighs its precision loss for Types D and E, producing higher F1 on these aggregation-oriented queries. These results suggest that flat selection is preferable for focused information needs, whereas aspect-based selection is better suited to comprehensive aggregation, where omitting relevant evidence is more costly than retaining additional context.

Aspect-aware generation produces complete answers with highly faithful citations. We evaluate generation using completeness and citation faithfulness. For a question q with required information

Model effects vary by query type

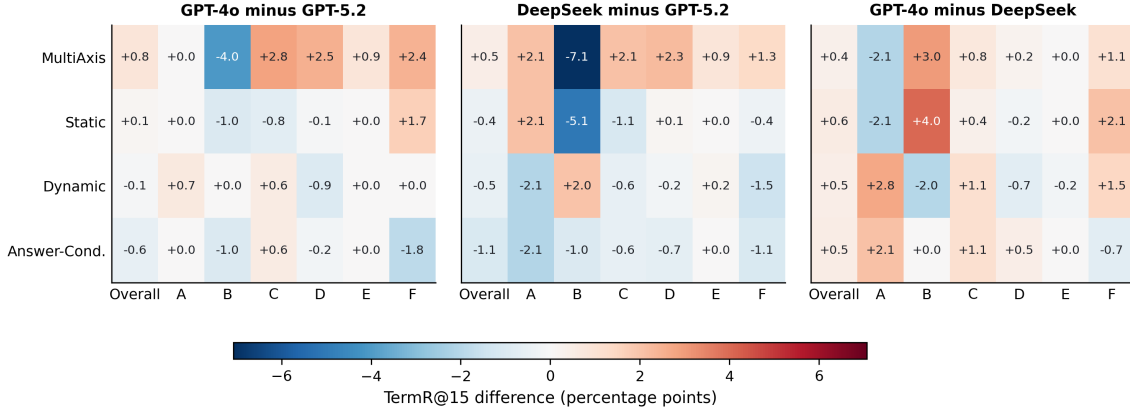


Figure 5: Type-wise TermR@15 differences between LLM backends. Positive values indicate that the model named first outperforms the model named second. Differences vary by query type and remain small overall.

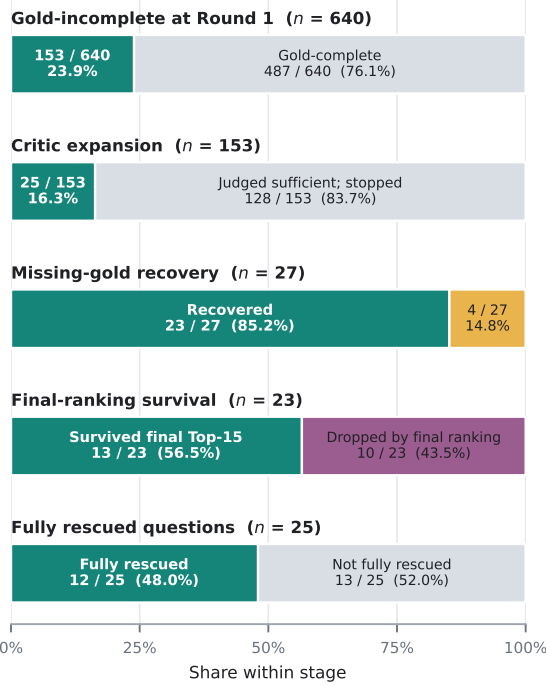


Figure 6: **Stage-wise decomposition of missing-evidence recovery.** The Dynamic Critic triggers expansion for only a small fraction of gold-incomplete Round 1 results. Once triggered, targeted requery recovers most missing gold threads, although final Top-15 reranking removes part of the recovered evidence. Percentages use the stage-specific denominator shown in each row.

points P_q , completeness is defined as

$$\text{Comp}(q) = \frac{1}{|P_q|} \sum_{p \in P_q} s(p), \quad s(p) \in \{0, 0.5, 1\},$$

where $s(p)$ indicates that a required point is not

Table 7: Source selection performance using the same final top-15 candidate threads. Recall measures the fraction of all gold threads selected, while precision measures the fraction of selected threads that are gold. Sel F1 is the harmonic mean of recall and precision.

Type	R@15	Sel Rec		Sel Prec		Sel F1	
		Flat	Aspect	Flat	Aspect	Flat	Aspect
A	0.82	0.77	0.79	0.66	0.59	0.71	0.68
B	0.89	0.83	0.87	0.76	0.72	0.79	0.78
C	0.95	0.88	0.93	0.81	0.72	0.84	0.81
D	0.91	0.75	0.85	0.69	0.65	0.72	0.73
E	0.98	0.81	0.89	0.87	0.84	0.83	0.87
F	0.73	0.52	0.62	0.41	0.35	0.46	0.45
All	0.85	0.73	0.80	0.65	0.60	0.69	0.68

covered, partially covered, or fully covered, respectively. Citation faithfulness is defined as

$$\text{Faith}(q) = \frac{\# \text{ supported cited claims in } q}{\# \text{ cited claims in } q}.$$

Both metrics are macro-averaged across questions; citation faithfulness is averaged over questions containing at least one cited claim.

As shown in Table 8, the final aspect-aware generator achieves an overall completeness of 0.81 and citation faithfulness of 0.96. Type C attains the highest scores on both metrics. Type F remains the most challenging, particularly in completeness, indicating that open-ended and relationally diffuse questions are more likely to omit required information. Nevertheless, the consistently high citation-faithfulness scores show that the generated claims are generally supported by the specific email threads they cite.

Table 8: Final generation performance. Critic denotes the direct draft from the Answer-Conditioned critic, while Top-15 uses the same final retrieved threads without selection. Citation faithfulness is unavailable for Critic because its drafts do not contain explicit citations.

Type	Completeness				Faithfulness		
	Critic	Top-15	Flat	Aspect	Top-15	Flat	Aspect
A	0.597	0.769	0.786	0.837	0.971	0.977	0.980
B	0.723	0.819	0.859	0.865	0.980	0.930	0.954
C	0.703	0.888	0.844	0.927	0.985	0.988	0.990
D	0.555	0.760	0.751	0.845	0.961	0.970	0.962
E	0.732	0.876	0.786	0.871	0.967	1.000	0.984
F	0.434	0.614	0.599	0.640	0.916	0.925	0.909
All	0.586	0.756	0.752	0.810	0.958	0.959	0.956

Table 9: Sufficiency false-positive and false-negative counts. Each entry reports FP/FN.

Type	Dynamic	Answer Cond.
A	19 / 15	16 / 41
B	10 / 1	7 / 4
C	5 / 2	5 / 5
D	42 / 9	43 / 19
E	2 / 6	1 / 16
F	55 / 6	54 / 12
All	133 / 39	126 / 97